

MODELLI E ARCHITETTURE AVANZATI DI BASI DI DATI

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 2° semestre

Indice

1	Data Intensive Applications	2
1.1	Building Blocks	2
1.2	Ruoli e Professioni nei Dati	2
1.3	Sistemi Operazionali vs Analitici	2
1.3.1	OLTP (Online Transaction Processing)	3
1.3.2	OLAP (Online Analytical Processing)	3
1.3.3	Confronto Sintetico	3
1.4	Data Warehouse	3
1.4.1	Il Processo ETL	4
1.5	Data Lake	4
1.5.1	Definizione e Principio Sushi	4
1.6	Cloud vs Self-Hosting	5
1.6.1	Self-Hosting (On-Premises)	5
1.6.2	Cloud Computing	5
1.7	Distributed Versus Single-Node Systems	5
1.7.1	Motivazioni per la Distribuzione	5
1.7.2	Sfide e Complessità	6
1.8	Data Systems, Law, and Society	6
1.9	Esempio Pratico: Tech Stack di Netflix	6
1.10	Case Study: Social Network Home Timelines	7
1.10.1	Requisiti e Carico di Lavoro	7
1.10.2	Approccio 1: Pull (Relazionale)	7
1.10.3	Approccio 2: Push (Materializzazione)	8
1.10.4	Approccio 3: Ibrido	8
1.11	Reliability, Scalability, Maintainability	8
1.11.1	Reliability (Affidabilità)	8
1.11.2	Scalability (Scalabilità)	9
1.11.3	Maintainability (Manutenibilità)	10

1 Data Intensive Applications

Oggi, molte applicazioni software sono classificate come **data-intensive** (ad alta intensità di dati), in contrapposizione a quelle **compute-intensive** (ad alta intensità di calcolo).

Data-intensive: Sono applicazioni in cui il collo di bottiglia non è la potenza della CPU, ma la gestione dei dati stessi. Le sfide principali riguardano:

- La **mole** dei dati (volume);
- La **complessità** dei dati;
- La **velocità** con cui i dati cambiano.

Queste applicazioni devono garantire consistenza, disponibilità e affidabilità anche in presenza di guasti. Esempi tipici sono i social network, piattaforme di e-commerce, sistemi bancari e piattaforme di telemetria IoT.

Compute-intensive: Sono applicazioni dove il collo di bottiglia è la pura potenza di elaborazione (CPU). In questi casi, il volume di dati può essere ridotto, ma le operazioni matematiche o logiche richieste sono estremamente complesse (es. rendering video, simulazioni scientifiche).

1.1 Building Blocks

La maggior parte delle applicazioni data-intensive non è monolitica, ma è costruita combinando diverse tecnologie specializzate:

- **Database:** Per immagazzinare e interrogare i dati in modo persistente.
- **Cache:** Per memorizzare risultati costosi o dati acceduti frequentemente in memoria volatile, velocizzando le letture.
- **Search Indexes:** Per consentire ricerche complesse (es. per parole chiave o filtri) che un database tradizionale gestirebbe lentamente.
- **Stream Processing:** Per l'elaborazione asincrona di messaggi e eventi in tempo reale.
- **Batch Processing:** Per elaborare periodicamente grandi volumi di dati accumulati nel tempo.

1.2 Ruoli e Professioni nei Dati

La gestione di sistemi complessi richiede diverse figure professionali:

- **Backend Engineers:** Costruiscono i servizi che gestiscono le richieste operative (lettura/scrittura) dell'applicazione.
- **Business Analysts:** Utilizzano i dati per generare report e metriche di *Business Intelligence* (BI) a supporto delle decisioni manageriali.
- **Data Scientists:** Esplorano i dati per trovare pattern o creare modelli di Machine Learning (es. sistemi di raccomandazione "chi ha comprato X ha comprato anche Y").
- **Data Engineers:** Sono responsabili dell'infrastruttura dati; integrano i sistemi operazionali con quelli analitici e gestiscono le pipeline di dati.
- **Analytics Engineers:** Curano la trasformazione e la modellazione dei dati per renderli puliti e utilizzabili da analisti e data scientist.

1.3 Sistemi Operazionali vs Analitici

Esiste una distinzione fondamentale tra i sistemi utilizzati per gestire l'operatività quotidiana e quelli usati per l'analisi dei dati.

1.3.1 OLTP (Online Transaction Processing)

Sono i sistemi che supportano l'operatività dell'applicazione (es. il database di un e-commerce).

- **Scopo:** Gestire transazioni rapide e frequenti eseguite dagli utenti.
- **Accesso:** Letture e scritture di piccoli record identificati da una chiave.
- **Stato dei dati:** Rappresentano lo stato corrente (es. "qual è il saldo attuale?").

1.3.2 OLAP (Online Analytical Processing)

Sono sistemi ottimizzati per l'analisi dei dati da parte di analisti e data scientist.

- **Scopo:** Rispondere a domande complesse sul business (es. "Ricavi totali per mese").
- **Accesso:** Aggregazioni su grandi quantità di record (scansione di molte righe, poche colonne).
- **Stato dei dati:** Rappresentano la storia degli eventi nel tempo.

1.3.3 Confronto Sintetico

Proprietà	OLTP (Operazionale)	OLAP (Analitico)
Pattern di Lettura	Query puntuali (cerca per chiave), pochi record.	Aggregazioni su grandi moli di dati.
Pattern di Scrittura	Accesso casuale, aggiornamenti utente in tempo reale.	Importazione massiva (Batch/-Stream).
Utenti	Clienti, Applicazioni Backend.	Analisti, Data Scientists, Manager.
Dati	Stato corrente, dati normalizzati.	Storico degli eventi, dati denormalizzati.
Dimensione	Gigabyte - Terabyte.	Terabyte - Petabyte.

Tabella 1: Differenze tra OLTP e OLAP

1.4 Data Warehouse

Un **Data Warehouse** (DW) è un database separato, ottimizzato per carichi di lavoro OLAP. L'utilizzo diretto dei database OLTP per l'analisi è sconsigliato per tre motivi:

1. **Isolamento delle performance:** Le query analitiche sono pesanti e potrebbero rallentare l'applicazione per gli utenti finali.
2. **Data Silos:** I dati sono spesso sparsi su più sistemi operazionali incompatibili tra loro.
3. **Schema:** Gli schemi OLTP sono normalizzati (ottimi per la coerenza in scrittura), mentre l'analisi richiede schemi denormalizzati (ottimi per la velocità di lettura).

All'interno di un Data Warehouse, i dati devono essere non normalizzati per evitare costose operazioni di JOIN durante l'analisi. La denormalizzazione è una pratica comune in OLAP, dove si accettano dati ridondanti per ottenere prestazioni migliori nelle query analitiche.

Normalizzazione è un processo di progettazione del database che mira a ridurre la ridondanza dei dati e migliorare l'integrità dei dati. In un database normalizzato, i dati sono organizzati in tabelle separate con relazioni ben definite, e le operazioni di lettura richiedono spesso JOIN tra queste tabelle.

1.4.1 Il Processo ETL

Per popolare un Data Warehouse si utilizza il processo **ETL**:

- **Extract (E)**: Estrazione dei dati dai vari sistemi sorgente (OLTP).
- **Transform (T)**: Pulizia e trasformazione dei dati. In questa fase avviene spesso la *denormalizzazione*, convertendo i dati in schemi a stella o a fiocco di neve (Star/Snowflake schema) per evitare costose operazioni di JOIN durante l'analisi.
- **Load (L)**: Caricamento dei dati nel Data Warehouse.

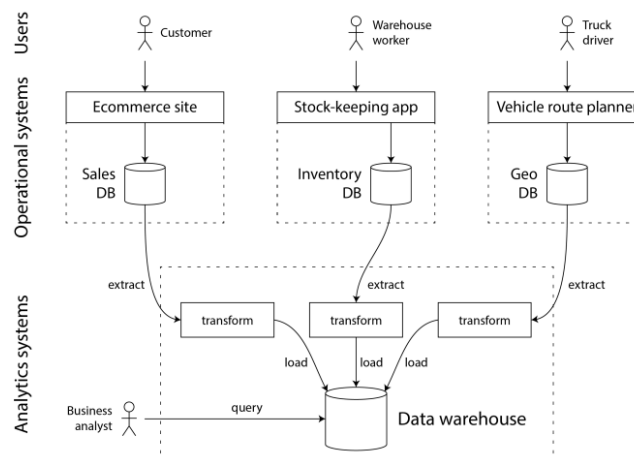


Figura 1: Schema del processo ETL: Dai sistemi operazionali al Data Warehouse

Varianti:

- **ELT**: In sistemi moderni potenti, si preferisce caricare i dati grezzi e trasformarli successivamente all'interno del Warehouse stesso.
- **CDC (Change Data Capture)**: Metodo di estrazione continua dove ogni modifica nel DB sorgente viene replicata quasi in tempo reale nel sistema di destinazione.

1.5 Data Lake

Il **Data Warehouse**, basandosi sul modello relazionale (SQL), ha dei limiti per le moderne applicazioni di Data Science e Machine Learning. I modelli di ML lavorano su vettori e matrici (features numeriche) e spesso richiedono l'analisi di dati **non strutturati** (testo, immagini, video, audio) che non si adattano alle rigide tabelle di un database.

1.5.1 Definizione e Principio Sushi

Un **Data Lake** è un repository centralizzato che contiene una copia di tutti i dati in formato **grezzo** (raw).

- **Contenuto**: File grezzi (CSV, JSON, immagini) o formati ottimizzati per i Big Data (Avro, Parquet).
- **Schema-on-read**: A differenza del Warehouse (Schema-on-write), nel Lake non si impone una struttura al momento del salvataggio. La struttura viene definita solo quando i dati vengono letti.

Il Principio Sushi: Il concetto alla base del Data Lake è che *"il crudo è meglio del cotto"*. Mantenere i dati grezzi offre la massima flessibilità: i dati possono essere trasformati in modi diversi in futuro per rispondere a domande che oggi non siamo ancora in grado di formulare, senza il rischio di aver perso informazioni durante una trasformazione prematura.

1.6 Cloud vs Self-Hosting

La scelta tra self-hosting e cloud dipende dalla gestione dell'infrastruttura e dalla variabilità del carico di lavoro.

1.6.1 Self-Hosting (On-Premises)

Il sistema è costruito su risorse di proprietà: sistema operativo general-purpose (Linux, Windows), filesystem, networking TCP/IP e hardware fisico (CPU, RAM) .

1.6.2 Cloud Computing

I servizi cloud sono particolarmente preziosi quando il carico sul sistema varia significativamente nel tempo. I modelli di servizio si differenziano per il livello di gestione affidato al vendor :

- **IaaS (Infrastructure as a Service):** Il vendor fornisce macchine virtuali (hardware virtuale). L'utente gestisce il sistema operativo, il middleware e le applicazioni .
- **PaaS (Platform as a Service):** Fornisce macchine virtuali e strumenti software per lo sviluppo. Il vendor gestisce anche il sistema operativo e il runtime .
- **SaaS (Software as a Service):** Il vendor fornisce un'applicazione pronta all'uso; l'utente gestisce solo l'accesso e l'utilizzo .
- **FaaS (Function as a Service / Serverless):** Consente di eseguire funzioni specifiche (es. transazioni di pagamento) senza gestire alcuna infrastruttura.

Esempi di sistemi:

Categoria	Self-hosted	Cloud-native
Operational (OLTP)	MySQL, PostgreSQL, MongoDB	Azure SQL, Google Spanner, AWS Aurora
Analytical (OLAP)	Teradata, ClickHouse, Spark	Snowflake, BigQuery, Azure Synapse

Tabella 2: Esempi di sistemi self-hosted e cloud-native per OLTP e OLAP

1.7 Distributed Versus Single-Node Systems

I **sistemi distribuiti** sono composti da più nodi che comunicano via rete.

1.7.1 Motivazioni per la Distribuzione

- **Tolleranza ai guasti:** La ridondanza permette al sistema di funzionare anche in caso di guasto a macchine o datacenter.
- **Scalabilità ed Elasticità:** Possibilità di gestire carichi superiori alla capacità di una singola macchina e scalare dinamicamente le risorse in base alla domanda (allineando costi e utilizzo).
- **Latenza:** Riduzione dei tempi di risposta tramite la distribuzione geografica dei servizi vicino agli utenti.
- **Hardware Specializzato:** Utilizzo di nodi ottimizzati per compiti specifici (es. GPU per Machine Learning, nodi ad alta densità di storage) .
- **Compliance e Sostenibilità:** Rispetto delle leggi sulla residenza dei dati (jurisdiction) e possibilità di schedare i carichi dove l'energia rinnovabile è più abbondante o economica .

1.7.2 Sfide e Complessità

L'architettura distribuita introduce criticità non presenti nei sistemi single-node:

- **Efficienza della comunicazione:** La comunicazione via rete è significativamente più lenta rispetto alle chiamate in-process. In alcuni casi, è più efficiente spostare la computazione verso la macchina che possiede già i dati.
- **Performance:** Un singolo sistema single-threaded potente può talvolta superare un cluster distribuito con oltre 100 core CPU.
- **Affidabilità della rete:** Incertezza sulla ricezione delle richieste e problemi di **idempotenza** (rischio di rieseguire operazioni non sicure in caso di retry) .
- **Observability:** La risoluzione dei problemi richiede strumenti avanzati di monitoraggio come log, metriche e tracing (es. OpenTelemetry).
- **Consistenza:** Mantenere i dati coerenti tra servizi diversi è intrinsecamente difficile.

1.8 Data Systems, Law, and Society

Le normative legali influenzano direttamente l'architettura tecnica:

- **Data Residency:** Le leggi possono imporre che i dati rimangano entro specifici confini nazionali.
- **Diritto alla cancellazione:** Questo richiede procedure tecniche rigorose per eliminare i dati non solo dai database attivi, ma anche dai sistemi di backup e storage a lungo termine, garantendo una cancellazione sicura e totale.

1.9 Esempio Pratico: Tech Stack di Netflix

Netflix utilizza un approccio ibrido e distribuito per gestire la complessità:

- **Database Relazionali:** MySQL per billing e sottoscrizioni; Amazon Aurora per consolidare l'infrastruttura SQL; CockroachDB per transazioni globali con consistenza multi-regione.
- **NoSQL:** Apache Cassandra per la cronologia di visione e raccomandazioni; EVCache (basato su Memcached) per il caching distribuito; Atlas per metriche operative time-series.
- **Data Lake:** Amazon S3 con Apache Iceberg per dati analitici e non strutturati (log, video metadata).

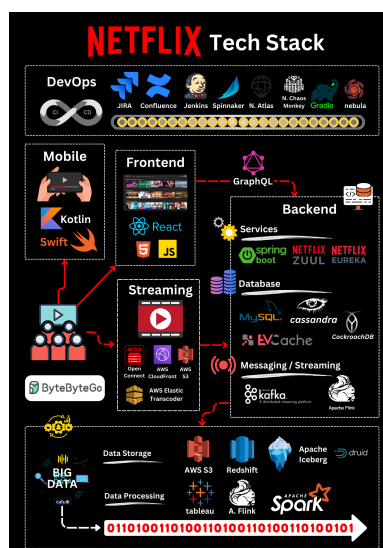


Figura 2: Esempio di Tech Stack distribuito e ibrido di Netflix

1.10 Case Study: Social Network Home Timelines

Prendiamo come esempio un social network come X (Twitter) per analizzare come i carichi di lavoro influenzino l'architettura.

1.10.1 Requisiti e Carico di Lavoro

I dati medi del sistema sono:

- **Post:** circa 5.700 post/secondo in media, con picchi che possono raggiungere i 150.000 post/-secondo.
- **Consultazione Timeline:** gli utenti consultano la propria timeline circa 2 milioni di volte al secondo.
- **Connessioni:** un utente medio segue 200 persone e ha 200 follower, ma esistono casi estremi come celebrità con oltre 100 milioni di follower.

1.10.2 Approccio 1: Pull (Relazionale)

In questo approccio, i post vengono salvati in una tabella globale e la timeline viene generata "al volo" ogni volta che un utente la richiede. Per implementare la timeline della home, il database utilizza la tabella *follows* per trovare tutti gli utenti seguiti dall'utente corrente, recupera i post recenti di quegli utenti e li ordina per timestamp.

Query per la generazione della timeline (Pull)

```
SELECT posts.*, users.* FROM posts
  JOIN follows ON posts.sender_id = follows.followee_id
  JOIN users ON posts.sender_id = users.id WHERE follows.follower_id =
    $CURRENT_USER$
ORDER BY posts.timestamp DESC
LIMIT 1000
```

- **Efficienza in scrittura:** Inserire un nuovo post è economico poiché richiede solo un semplice inserimento nella collezione globale dei post.
- **Costo in lettura:** Caricare la timeline con questa query è estremamente costoso.
- **Impatto sul sistema:** Se un utente segue 200 persone, il sistema deve recuperare e unire le liste dei post di ciascuna di esse. Con 2 milioni di query al secondo, il database dovrebbe eseguire circa 400 milioni di lookup di post al secondo.
- **Scalabilità:** Per gli utenti che seguono decine di migliaia di account, questa query è difficilmente ottimizzabile e molto lenta da eseguire.

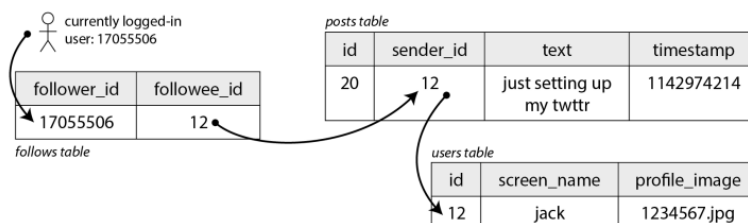


Figura 3: Semplice schema relazionale per l'implementazione di una timeline della home di X

1.10.3 Approccio 2: Push (Materializzazione)

Per ovviare ai costi di lettura, si possono precalcolare i risultati e servirli tramite una cache.

- **Materializzazione:** Consiste nel precalcolare e aggiornare i risultati di una query; la cache della timeline è un esempio di **vista materializzata**.
- **Fan-out:** Al momento della pubblicazione, il post viene consegnato alla cache di ogni follower.
- **Confronto prestazioni:** Con 5.700 post al secondo e 200 follower medi, si effettuano circa 1 milione di scritture al secondo, un risparmio significativo rispetto ai 400 milioni di lookup dell'approccio pull.
- **Problema degli Influencer:** Per gli utenti con milioni di follower (VIP), il lavoro di inserimento simultaneo in milioni di timeline diventa eccessivo.

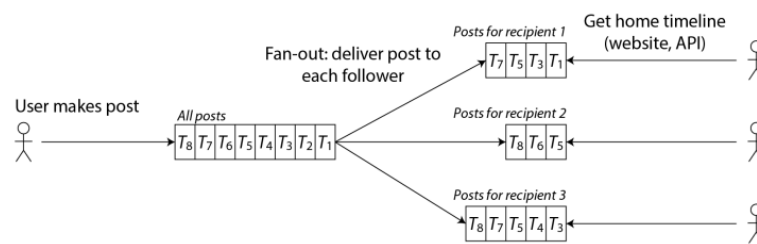


Figura 4: Consegnare i nuovi post a tutti i follower

1.10.4 Approccio 3: Ibrido

La soluzione più efficiente prevede un modello misto:

- Si usa la **materializzazione** (push) per gli utenti comuni.
- Si usa l'**approccio originale** (pull) per i post degli influencer, unendoli alla timeline materializzata solo al momento della lettura.

Lezione appresa: Non esiste un modello di dati perfetto in assoluto; è fondamentale scegliere il modello che meglio si allinea ai **pattern di accesso** ai dati del sistema specifico.

1.11 Reliability, Scalability, Maintainability

Nello sviluppo di sistemi software, e in particolare di applicazioni data-intensive, tre concetti sono di primaria importanza: l'affidabilità, la scalabilità e la manutenibilità.

1.11.1 Reliability (Affidabilità)

Un sistema è considerato affidabile se continua a funzionare correttamente anche in presenza di guasti o errori umani. È fondamentale distinguere tra:

- **Fault (Guasto):** Quando una singola componente del sistema devia dalle sue specifiche.
- **Failure (Fallimento):** Quando il sistema nel suo complesso smette di fornire il servizio all'utente.

L'obiettivo non è prevenire ogni singolo guasto (impossibile), ma progettare meccanismi di **fault-tolerance** per evitare che i guasti si trasformino in fallimenti.

Le principali fonti di guasto includono:

- **Guasti Hardware:** I guasti ai singoli dischi o macchine sono la norma su larga scala (es. in un cluster con 10.000 dischi, in media uno muore ogni giorno). La difesa consiste nella ridondanza (RAID, backup, alimentatori doppi) e in software capaci di tollerare la perdita di intere macchine.

- **Guasti Software:** Più difficili da prevedere rispetto all'hardware, sono spesso sistematici (es. bug nel kernel, memory leak, race conditions). Possono causare *fallimenti a cascata* nei sistemi distribuiti.
- **Errori Umani:** Gli studi indicano che la maggior parte dei periodi di inattività è causata da errori di configurazione o operativi degli esseri umani. Si combattono con buone UI, sandbox per i test e la capacità di effettuare rapidi **rollback** delle modifiche.

1.11.2 Scalability (Scalabilità)

La scalabilità è la capacità del sistema di gestire la crescita, sia essa in termini di volume di dati, carico di traffico o complessità delle operazioni.

Performance vs Scalabilità Mentre la **performance** indica quanto velocemente il sistema risponde con un carico fisso, la **scalabilità** descrive come il sistema si comporta all'aumentare del carico. Un sistema scalabile mantiene buone prestazioni anche quando il carico cresce.

Metriche di Prestazione

- **Response Time:** Il tempo totale visto dal client, inclusi ritardi di rete e code.
- **Service Time:** Il tempo effettivo durante il quale il servizio sta processando la richiesta.
- **Queueing Delays (Ritardi di Coda):** Possono verificarsi in diversi punti del flusso, ad esempio:
 - Attesa che una CPU si liberi per iniziare l'elaborazione.
 - Bufferizzazione dei pacchetti di risposta (es. TCP buffer) prima dell'invio se l'interfaccia di rete è congestionata da altri task.
- **Latency (Latenza):** Il tempo in cui una richiesta è "latente", ovvero non viene attivamente processata (es. ritardi in coda o in rete).
- **Throughput:** Numero di operazioni al secondo; metrica vitale per i sistemi *batch*.

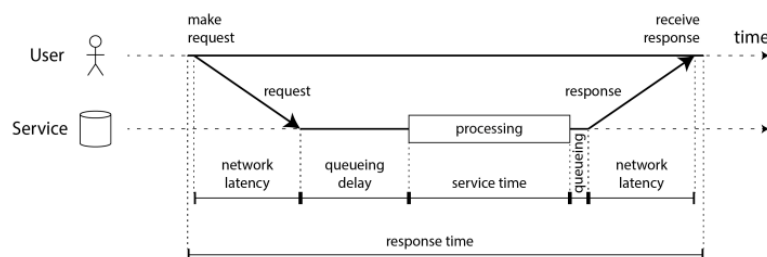


Figura 5: Diagramma la latenza e il tempo di risposta

Per misurare correttamente i tempi di risposta, non si usano le medie, ma i **percentili**:

$X^{th} percentile = Y ms$ significa che il $X\%$ delle richieste ha un tempo di risposta inferiore a $Y ms$

- **Mediana (p50):** Indica il tempo di risposta tipico per l'utente.
- **p95, p99 (Tail Latencies):** Rappresentano i casi peggiori. Ottimizzare il **p99.9** è cruciale per aziende come Amazon, poiché gli utenti con più dati (i più alto spendenti) sono spesso quelli che subiscono le latenze peggiori.

Esempio: L'impatto economico delle Tail Latencies (Amazon) Gli utenti che sperimentano le latenze peggiori (tail latencies) sono spesso i clienti più preziosi, poiché possiedono più dati e hanno effettuato più acquisti. Amazon ha osservato che:

- Un aumento di soli **100 ms** nel tempo di risposta riduce le vendite dell'**1%**.
- L'ottimizzazione del **99.99-esimo percentile** (la richiesta più lenta su 10.000) è stata ritenuta troppo costosa in termini di risorse rispetto al beneficio economico marginale, portando l'azienda a concentrarsi sul 99.9-esimo percentile.

Scaling Verticale vs Orizzontale

- **Verticale (Scaling Up):** Acquistare una macchina più potente. Semplice ma limitato dall'hardware.
- **Orizzontale (Scaling Out):** Aggiungere più macchine al cluster. Essenziale per sistemi web su larga scala.
- **Elasticità:** Capacità di alcuni sistemi cloud di aggiungere o rimuovere automaticamente risorse in base al carico corrente.

1.11.3 Maintainability (Manutenibilità)

La maggior parte del costo di un software risiede nella sua manutenzione continua, non nello sviluppo iniziale. Un sistema manutenibile deve essere facile da operare, comprendere e adattare.

I tre principi cardine del design sono:

1. **Operability (Operatività):** Rendere i compiti di routine semplici e sicuri per i team operativi.
2. **Simplicity (Semplicità):** Rimuovere la "complessità accidentale" per rendere il sistema comprensibile.
3. **Evolvability (Evolvibilità):** Permettere modifiche sicure al sistema per adattarsi a nuovi requisiti senza riscritture massicce.